Collection of plots

Airas Group

RStudio, PBC

2025/5/9 (updated: 2025-11-05)

Plots for exploratory analysis

1. Boxplots

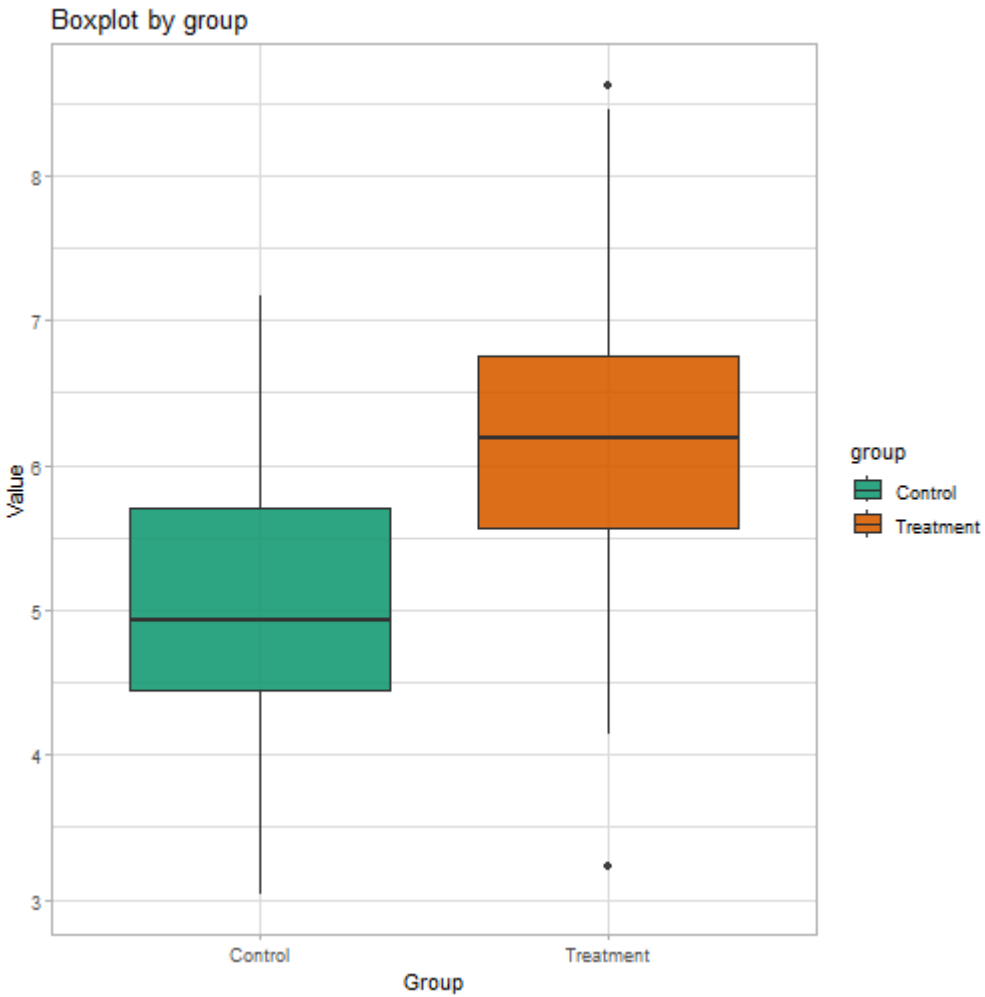
```r
library(ggplot2)
library(ggpubr)
library(ggdist)  #for half-eye / raincloud plots

set.seed(123) #for the reproducibility of generated data
df <- data.frame(
  group = rep(c("Control", "Treatment"), each = 50),
  value = c(rnorm(50, mean=5, sd=1),
            rnorm(50, mean=6, sd=1.2))
)
head(df)
```

```
##     group    value
## 1 Control 4.439524
## 2 Control 4.769823
## 3 Control 6.558708
## 4 Control 5.070508
## 5 Control 5.129288
## 6 Control 6.715065
```

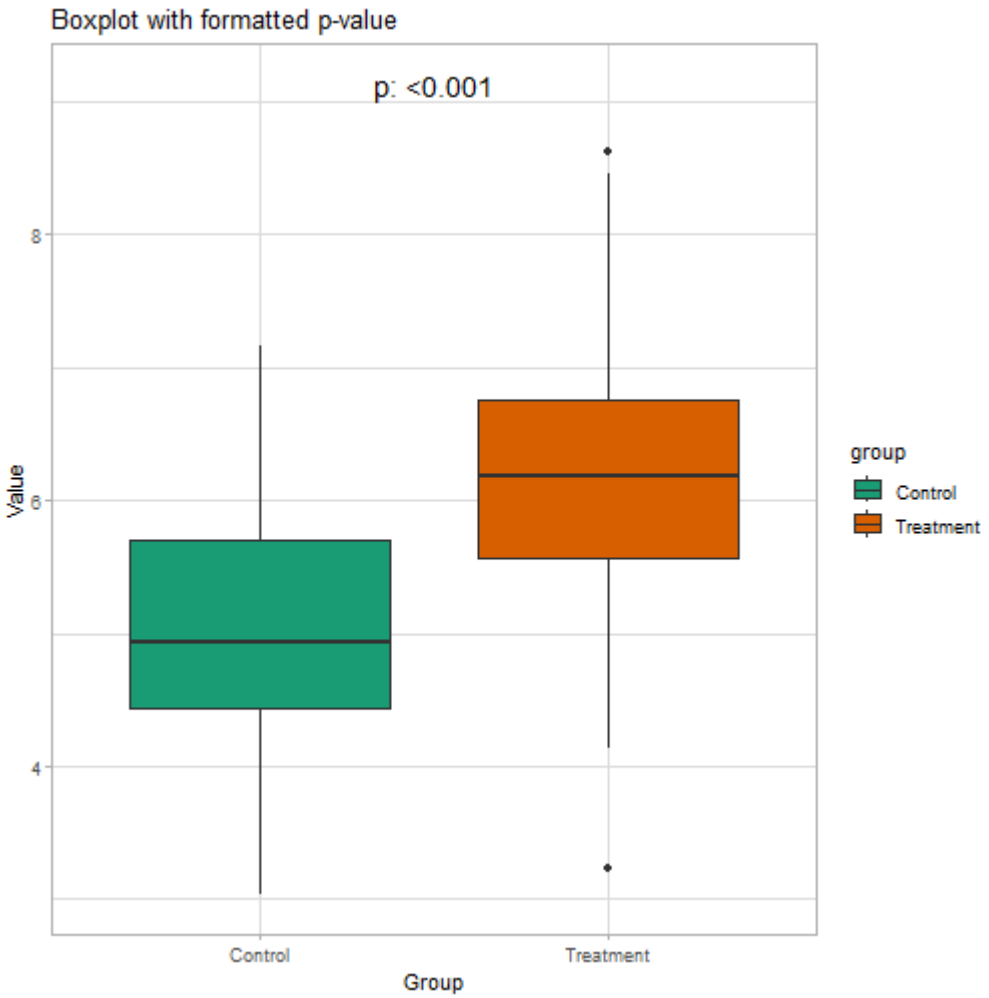
Classic boxplot

- Run the code:



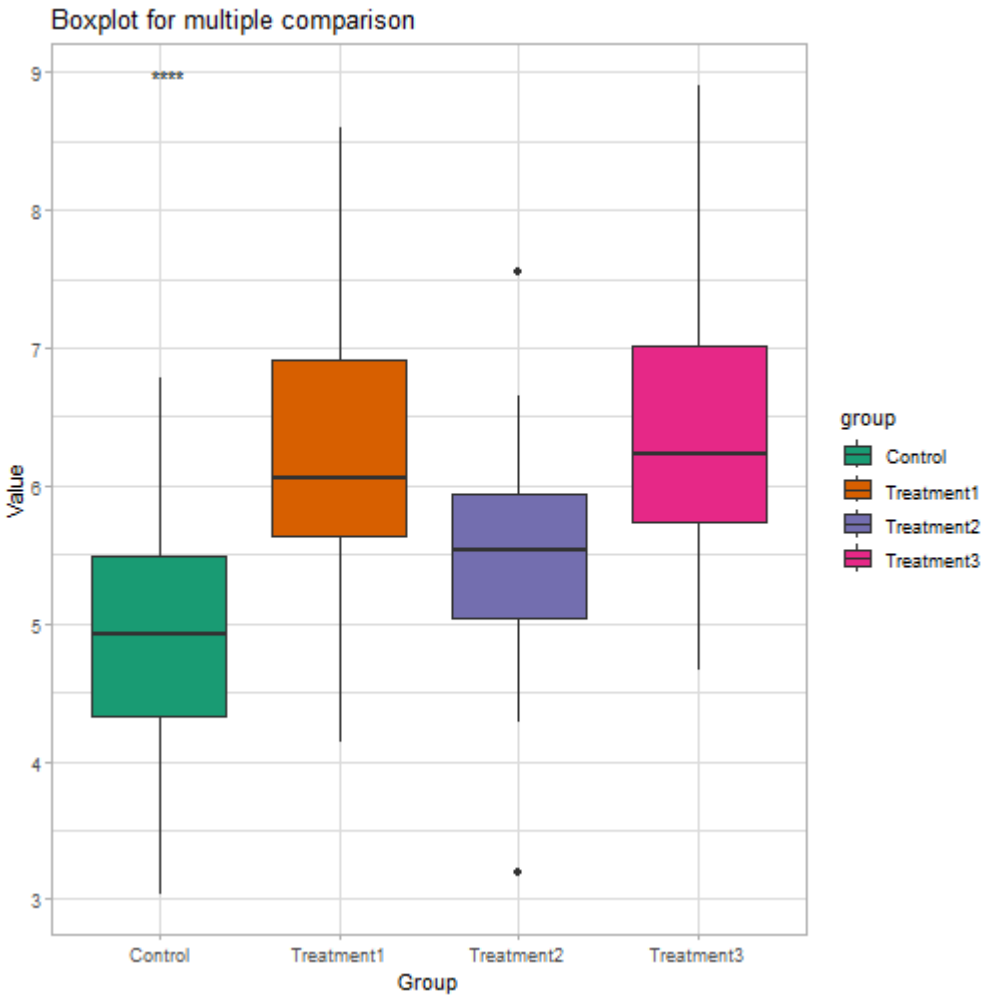
Boxplot with statistics

- Run the code:



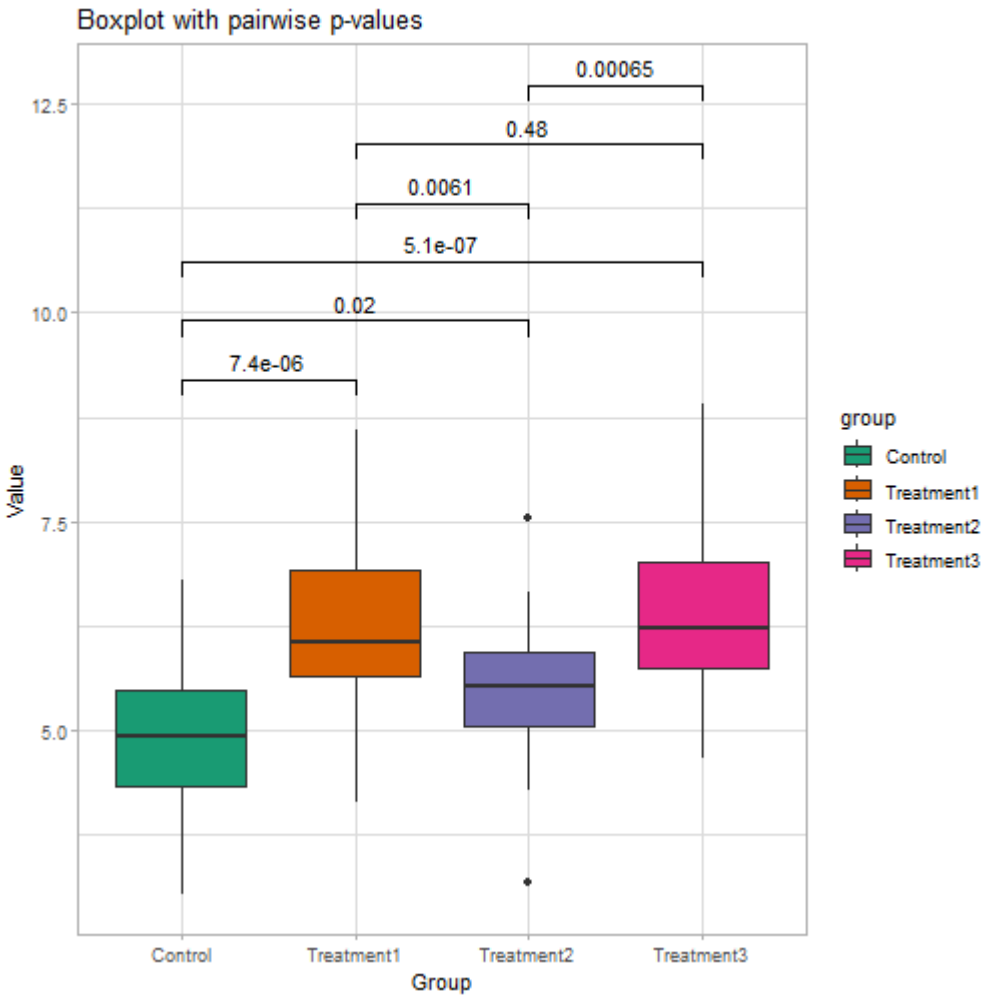
Group-wise comaprison

- Run the code:



Multiple comparisons

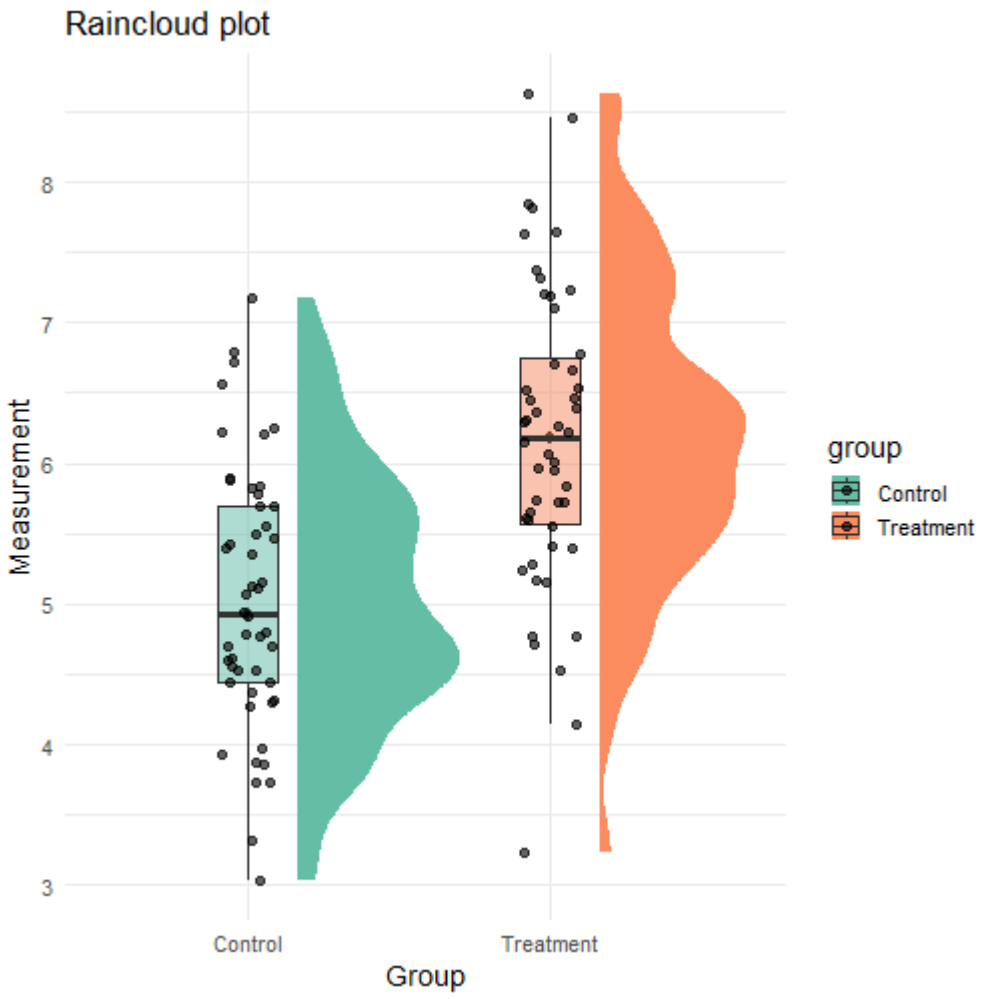
- Run the code:



2. Raincloud plots

We want to show boxplot, disperison of values and density of a sampled numeric variables, highlighitng the difference of distribution between two groups

- Run the code:


Raincloud plot
Measurement
8
7
6
5
4
3
Control
Treatment
Group
group
Control
Treatment

Plots for model estimates

1. Dot-and-whisker plot

Dot-and-whisker plots visually summarize regression results by showing the point estimate (dot) and confidence interval (whiskers) for each predictor. They are useful for quickly comparing effect sizes and significance across multiple variables in a single, clean plot.

- We start from running logistic regression model
- We obtain estimate (with p-values, confidence intervals) for each of the regressor in the model

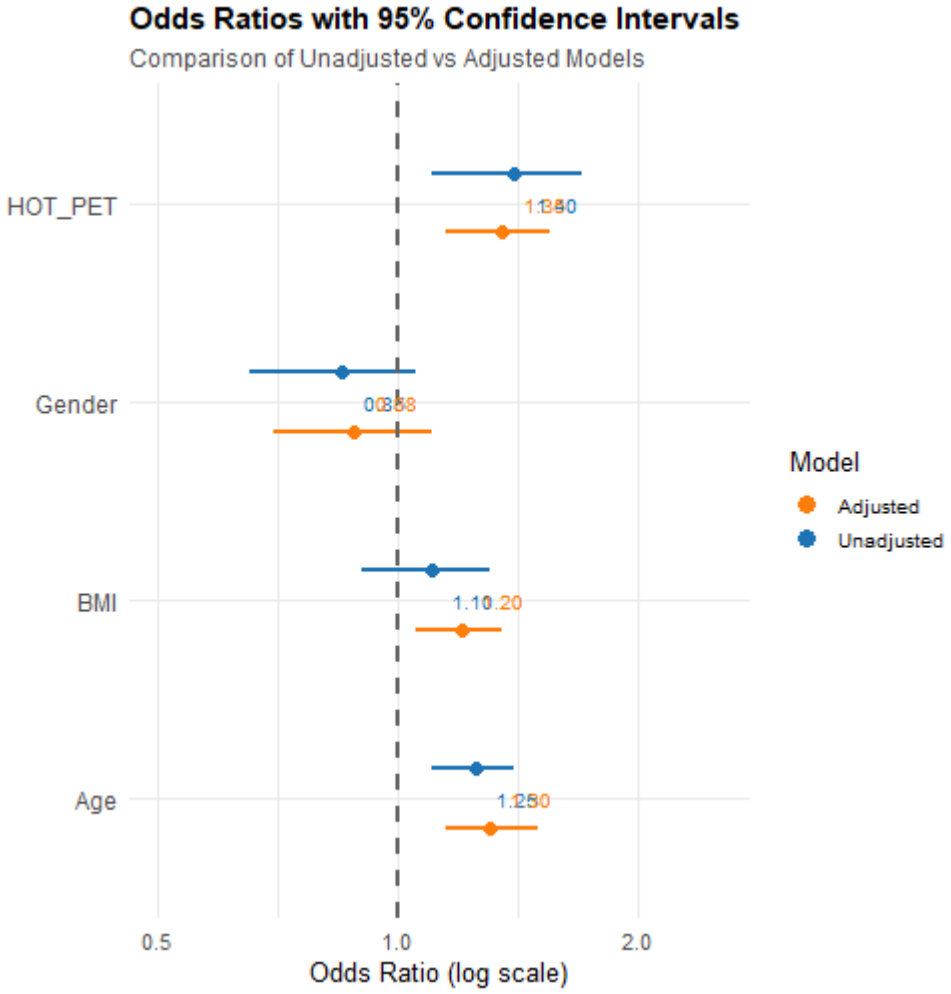
```
##   Predictor   OR CI_low CI_high p_value      model
## 1       Age 1.25   1.10    1.40   0.010 Unadjusted
## 2       Age 1.30   1.15    1.50   0.020   Adjusted
## 3    Gender 0.85   0.65    1.05   0.040 Unadjusted
## 4    Gender 0.88   0.70    1.10   0.030   Adjusted
## 5       BMI 1.10   0.90    1.30   0.120 Unadjusted
## 6       BMI 1.20   1.05    1.35   0.100   Adjusted
## 7   HOT_PET 1.40   1.10    1.70   0.001 Unadjusted
## 8   HOT_PET 1.35   1.15    1.55   0.002   Adjusted
```

- Now we need to arrange the data before plotting
- From the model output (logistic regression) we need to run this:

```
dw_data <- results_clean %>%

  rename(
    estimate = OR,
    conf.low = CI_low,
    conf.high = CI_high,
    term = Predictor
  ) %>%
  mutate(
    model = factor(model, levels = c("Unadjusted", "Adjusted")),
    term = fct_rev(factor(term)),
    label = sprintf("%.2f", estimate),
    sig = ifelse(p_value < 0.05, "Significant", "Not significant")
  )
```

- And on the dw_data arranged for the plot, we run the dwplot function:



Plots for accuracy/ frequency tables

1. Mosaic plot

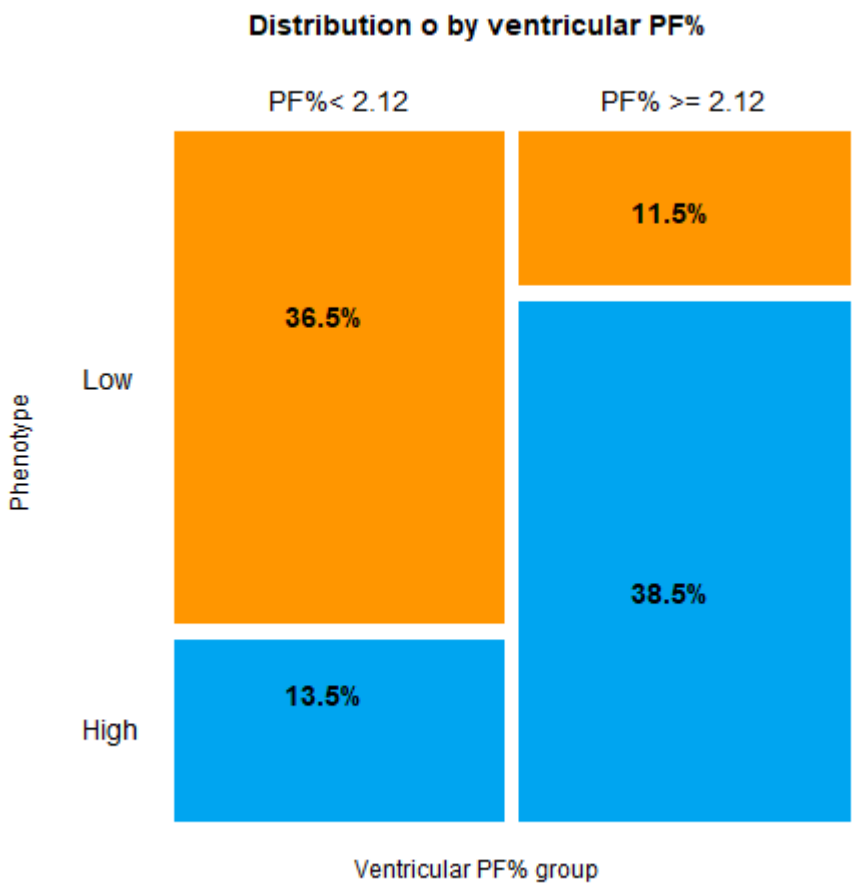
Mosaic plots are used to visualize contingency tables. The size of each tile represents the proportion of observations in each group-category combination, allowing for quick comparisonof relative frequencies.

- We estart from a contingency table as the following one:

```
mat <- matrix(c(73, 27, 23, 77),
              nrow = 2,
              byrow = TRUE,
              dimnames = list(
                Group = c("PF%< 2.12", "PF% >= 2.12"),
                Category = c("Low", "High")
              ))
mat
```

```
##                Category
## Group           Low High
##   PF%< 2.12      73   27
##   PF% >= 2.12    23   77
```

- Next we run the mosaicplot function:



Plots for longitudinal data

1. Spaghetti plot

A spaghetti plot is a type of line plot used to visualize longitudinal or repeated-measures data for multiple subjects. The key idea is to plot each subject's trajectory over time, so you can see both individual trends and the overall pattern.

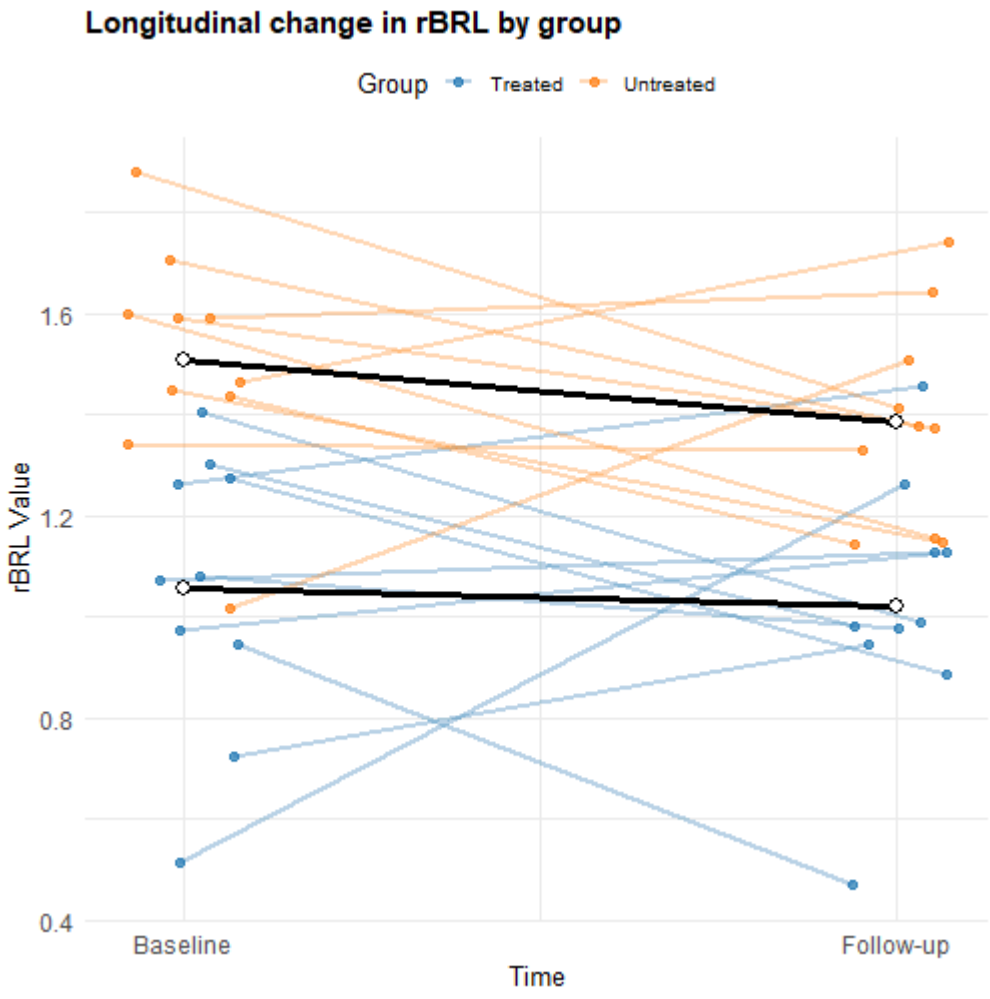
- We start from a dataset for example with the following variables:

```
set.seed(42) # the seed allows to generate the same values for rBRLs
 df <- data.frame(
   ID = rep(1:20, each = 2),
   Group = rep(c("Treated", "Untreated"), each = 20),
   Time = rep(c("baseline", "follow-up"), times = 20),
   rBRL = c(rnorm(20, 1, 0.2), rnorm(20, 1.5, 0.2))
 )
```

- Since some values will coincide, we would like to randomly jitter them (spiking in the 2d-space)
- We will prepare the data before plotting:

```
set.seed(42) # the seed allows to generate the same plot
df_jittered <- df %>%
  mutate(
    Time_num = as.numeric(factor(Time, levels = c("baseline", "follov
    Time_jittered = Time_num + runif(n(), -0.08, 0.08) # jitter for
  )
```

- Before plotting calculate the means:

```
# Group means (without jitter)
group_means <- df %>%
  group_by(Group, Time) %>%
  summarise(mean_rBRL = mean(rBRL), .groups = "drop") %>%
  mutate(Time_num = as.numeric(factor(Time, levels = c("baseline", "f
```

- Run entirely the code